ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

| ассистент | | К.А. Кочин |
|---|---|---|
| должность, уч. степень, звание | подпись, дата | инициалы, фамилия |

# ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

## Работа с буфером глубины, прозрачностью.

по курсу: КОМПЬЮТЕРНАЯ ГРАФИКА

РАБОТУ ВЫПОЛНИЛ

| СТУДЕНТ ГР. № | 4236 | | А.В. Панченко |
|---|---|---|---|
| | | подпись, дата | инициалы, фамилия |

Санкт-Петербург 2023

## 1. Задание

Вывести несколько пересекающихся объектов, как минимум один из них должен быть прозрачным. Включить 3 источника света, задать свойства поверхностей и источников. Организовать раздельное вращение источников света вокруг неподвижной сцены, управляемое с клавиатуры или мышью.

$$R = (R1*A1+R2*A2*(1 – A2)) / A$$

$$G = (G1*A1+G2*A2*(1 – A2)) / A$$

$$B = (B1*A1+B2*A2*(1 – A2)) / A$$

$$A = A1+A2*(1 - A1)$$

## 2. Текст программы

```
#include <iostream>
#include <glut.h>

// Угол поворота куба и пирамиды
float angleX = 0.0f;
float angleY = 0.0f;
int prevMouseX, prevMouseY;
bool isRotating = false;
bool isDragging = false;
int xDragStart = 0;
int yDragStart = 0;
GLfloat lightAngle = 0.0;
// Для первого домика
GLfloat light_position1[] = { -4.5f, 0.6f, 1.5f, 1.0f };

// Для второго домика
GLfloat light_position2[] = { 0.0f, 0.6f, 1.5f, 1.0f };

// Для третьего домика
GLfloat light_position3[] = { 4.5f, 0.6f, 1.5f, 1.0f };
enum ReflectionType {
    DIFFUSE,
    MIRROR,
    AMBIENT
};

void drawHouse(float roofColor[3], float wallColor[3], ReflectionType
reflectionType) {
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

    // Основание домика (квадрат)
    glBegin(GL_QUADS);
    glColor3fv(wallColor); // Цвет пола
    glVertex3f(-1.0f, -1.0f, -1.0f);
    glVertex3f(1.0f, -1.0f, -1.0f);
    glVertex3f(1.0f, -1.0f, 1.0f);
    glVertex3f(-1.0f, -1.0f, 1.0f);
    glEnd();

    // Стены домика (четыре стены)
    glBegin(GL_QUADS);
    GLfloat wallAmbient[] = { wallColor[0], wallColor[1], wallColor[2], 1.0f }; //
Амбиентный цвет для стен
```

```
    GLfloat wallDiffuse[] = { wallColor[0], wallColor[1], wallColor[2], 1.0f }; //
Диффузный цвет для стен
    GLfloat wallSpecular[] = { 0.0f, 0.0f, 0.0f, 1.0f }; // Зеркальный цвет для стен
    GLfloat wallShininess[] = { 0.0f }; // Степень отражения для стен

    if (reflectionType == MIRROR) {
        // Для зеркального отражения
        wallAmbient[3] = 0.0f; // Отключаем амбиентное отражение
        wallDiffuse[3] = 0.5f; // Устанавливаем прозрачность для диффузного
отражения
        wallSpecular[0] = 1.0f;
        wallSpecular[1] = 1.0f;
        wallSpecular[2] = 1.0f;
        wallSpecular[3] = 1.0f;
        wallShininess[0] = 100.0f;
        glEnable(GL_BLEND);
        glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    }

    if (reflectionType == AMBIENT) {
        // Для смешанного отражения
        wallAmbient[3] = 1.0f; // Включаем амбиентное отражение
        wallDiffuse[3] = 0.0f; // Отключаем диффузное отражение
        wallSpecular[0] = 0.0f; // Отключаем зеркальное отражение
        wallSpecular[1] = 0.0f;
        wallSpecular[2] = 0.0f;
        wallSpecular[3] = 0.0f;
        wallShininess[0] = 0.0f; // Отключаем зеркальный блеск
    }
    else if (reflectionType == DIFFUSE) {
        wallAmbient[3] = 0.0f; // Отключаем амбиентное отражение
        wallDiffuse[3] = 1.0f; // Включаем диффузное отражение
        wallSpecular[0] = 0.0f; // Отключаем зеркальное отражение
        wallSpecular[1] = 0.0f;
        wallSpecular[2] = 0.0f;
        wallSpecular[3] = 0.0f;
        wallShininess[0] = 0.0f; // Отключаем зеркальный блеск
    }

    glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, wallAmbient);
    glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, wallDiffuse);
    glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, wallSpecular);
    glMaterialfv(GL_FRONT_AND_BACK, GL_SHININESS, wallShininess);

    glColor3fv(wallColor); // Цвет стен
    glVertex3f(-0.8f, -1.0f, -0.8f);
    glVertex3f(0.8f, -1.0f, -0.8f);
    glVertex3f(0.8f, 0.6f, -0.8f);
    glVertex3f(-0.8f, 0.6f, -0.8f);

    glColor3fv(wallColor); // Цвет стен
    glVertex3f(-0.8f, -1.0f, 0.8f);
    glVertex3f(0.8f, -1.0f, 0.8f);
    glVertex3f(0.8f, 0.6f, 0.8f);
    glVertex3f(-0.8f, 0.6f, 0.8f);

    glColor3fv(wallColor); // Цвет стен
    glVertex3f(-0.8f, -1.0f, -0.8f);
    glVertex3f(-0.8f, -1.0f, 0.8f);
    glVertex3f(-0.8f, 0.6f, 0.8f);
    glVertex3f(-0.8f, 0.6f, -0.8f);

    glColor3fv(wallColor); // Цвет стен
    glVertex3f(0.8f, -1.0f, -0.8f);
    glVertex3f(0.8f, -1.0f, 0.8f);
```

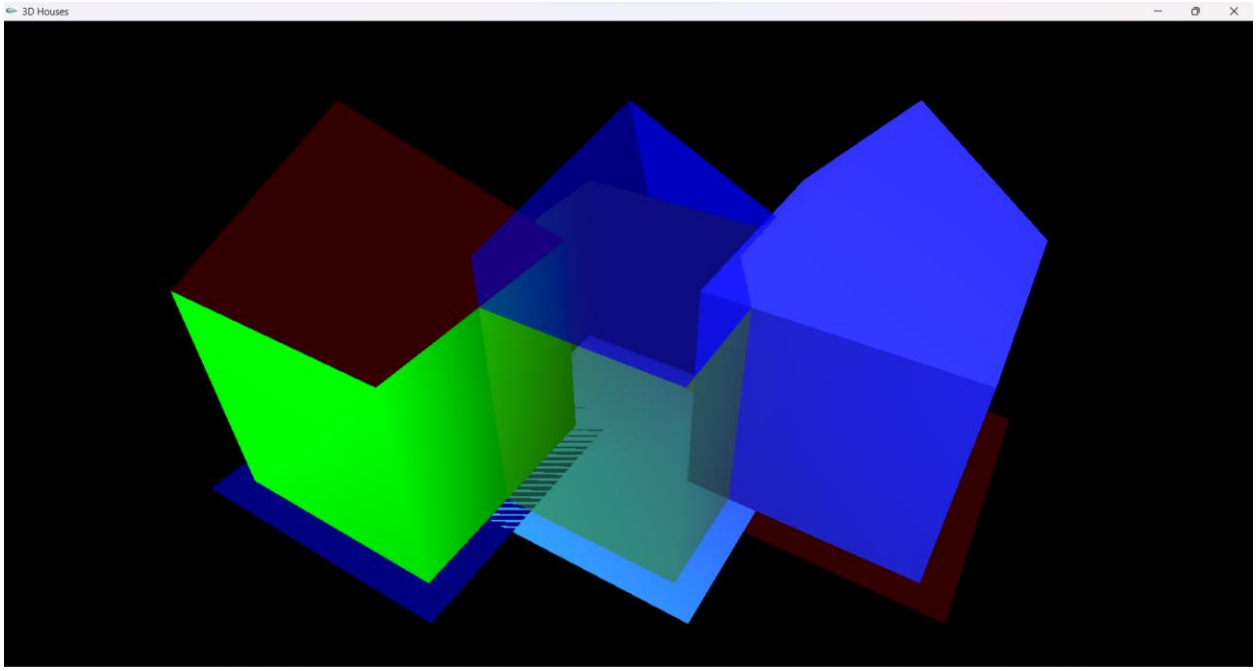
```
    glVertex3f(0.8f, 0.6f, 0.8f);
    glVertex3f(0.8f, 0.6f, -0.8f);
    glEnd();


    // Крыша (пирамида)
    glBegin(GL_TRIANGLES);
    GLfloat roofAmbient[] = { roofColor[0], roofColor[1], roofColor[2], 1.0f }; //
Амбиентный цвет для стен
    GLfloat roofDiffuse[] = { roofColor[0], roofColor[1], roofColor[2], 1.0f }; //
Диффузный цвет для стен
    GLfloat roofSpecular[] = { 0.0f, 0.0f, 0.0f, 1.0f }; // Зеркальный цвет для стен
    GLfloat roofShininess[] = { 0.0f }; // Степень отражения для стен

    if (reflectionType == MIRROR) {
        // Для зеркального отражения
        roofAmbient[3] = 0.0f; // Отключаем амбиентное отражение
        roofDiffuse[3] = 0.5; // Устанавливаем прозрачность для диффузного отражения
        roofSpecular[0] = 1.0f;
        roofSpecular[1] = 1.0f;
        roofSpecular[2] = 1.0f;
        roofSpecular[3] = 1.0f;
        roofShininess[0] = 100.0f;
        glEnable(GL_BLEND);
        glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    }

    if (reflectionType == AMBIENT) {
        // Для смешанного отражения
        roofAmbient[3] = 1.0f; // Включаем амбиентное отражение
        roofDiffuse[3] = 0.0f; // Отключаем диффузное отражение
        roofSpecular[0] = 0.0f; // Отключаем зеркальное отражение
        roofSpecular[1] = 0.0f;
        roofSpecular[2] = 0.0f;
        roofSpecular[3] = 0.0f;
        roofShininess[0] = 0.0f; // Отключаем зеркальный блеск
    }
    else if (reflectionType == DIFFUSE) {
        roofAmbient[3] = 0.0f; // Отключаем амбиентное отражение
        roofDiffuse[3] = 1.0f; // Включаем диффузное отражение
        roofSpecular[0] = 0.0f; // Отключаем зеркальное отражение
        roofSpecular[1] = 0.0f;
        roofSpecular[2] = 0.0f;
        roofSpecular[3] = 0.0f;
        roofShininess[0] = 0.0f; // Отключаем зеркальный блеск
    }

    glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, roofAmbient);
    glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, roofDiffuse);
    glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, roofSpecular);
    glMaterialfv(GL_FRONT_AND_BACK, GL_SHININESS, roofShininess);

    glColor3fv(roofColor); // Цвет крыши

    glColor3fv(roofColor); // Цвет крыши
    glVertex3f(-0.8f, 0.6f, -0.8f);
    glVertex3f(0.8f, 0.6f, -0.8f);
    glVertex3f(0.0f, 1.6f, 0.0f);

    glColor3fv(roofColor); // Цвет крыши
    glVertex3f(-0.8f, 0.6f, 0.8f);
    glVertex3f(0.8f, 0.6f, 0.8f);
    glVertex3f(0.0f, 1.6f, 0.0f);

    glColor3fv(roofColor); // Цвет крыши
```

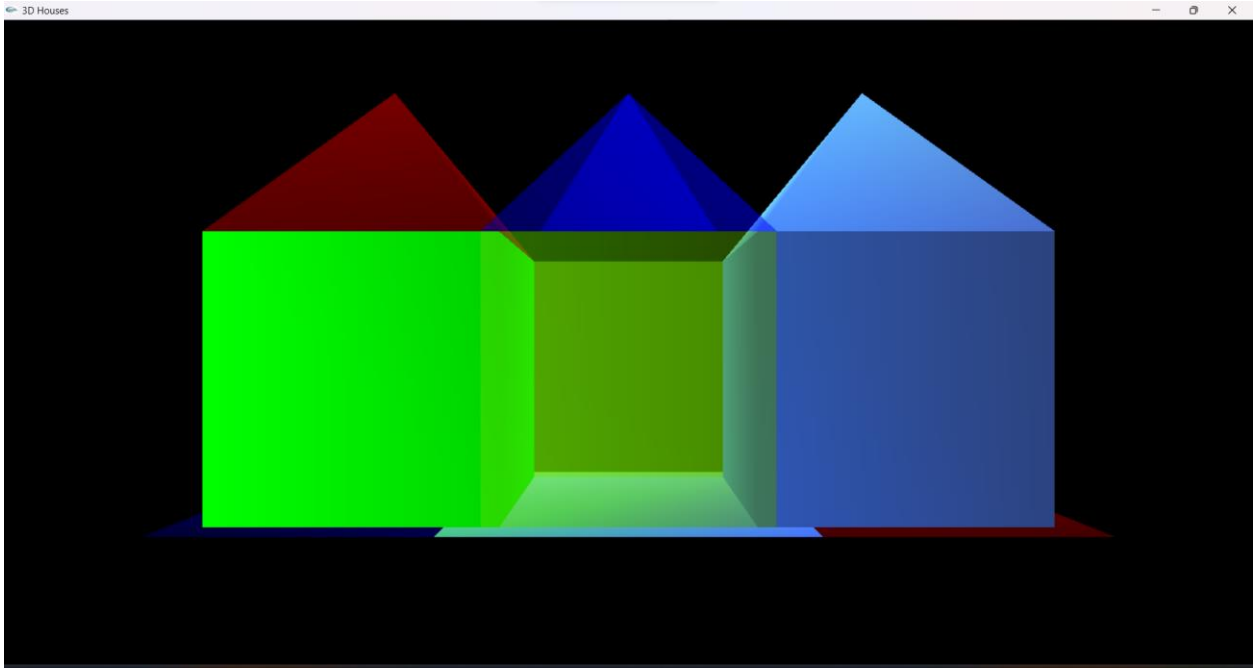
```
    glVertex3f(-0.8f, 0.6f, -0.8f);
    glVertex3f(0.0f, 1.6f, 0.0f);
    glVertex3f(-0.8f, 0.6f, 0.8f);

    glColor3fv(roofColor); // Цвет крыши
    glVertex3f(0.8f, 0.6f, -0.8f);
    glVertex3f(0.0f, 1.6f, 0.0f);
    glVertex3f(0.8f, 0.6f, 0.8f);
    glEnd();
}

void init() {
    glEnable(GL_LIGHTING);
    glEnable(GL_DEPTH_TEST);
    glDepthFunc(GL_LEQUAL);

    // Устанавливаем глобальные параметры фонового освещения
    GLfloat ambientLight[] = { 0.2f, 0.2f, 0.2f, 1.0f }; // Изменено на
положительные значения
    glLightfv(GL_LIGHT0, GL_AMBIENT, ambientLight);

    GLfloat lightIntensity1[] = { 1.5f, 0.0f, 0.0f, 1.0f }; // Красный свет
    GLfloat lightPosition1[] = { -8.0f, 2.0f, 0.0f, 1.0f }; // Красный свет
(относительно первого домика)
    glLightfv(GL_LIGHT1, GL_DIFFUSE, lightIntensity1);
    glLightfv(GL_LIGHT1, GL_POSITION, light_position1);

    GLfloat lightIntensity2[] = { 0.0f, 1.5f, 0.0f, 1.0f }; // Зеленый свет
    GLfloat lightPosition2[] = { -2.0f, 2.0f, 0.0f, 1.0f }; // Зеленый свет
(относительно второго домика)
    glLightfv(GL_LIGHT2, GL_DIFFUSE, lightIntensity2);
    glLightfv(GL_LIGHT2, GL_POSITION, light_position2);

    GLfloat lightIntensity3[] = { 0.0f, 0.0f, 1.5f, 1.0f }; // Синий свет
    GLfloat lightPosition3[] = { 4.0f, 2.0f, 0.0f, 1.0f }; // Синий свет
(относительно третьего домика)
    glLightfv(GL_LIGHT3, GL_DIFFUSE, lightIntensity3);
    glLightfv(GL_LIGHT3, GL_POSITION, light_position3);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    glTranslatef(0.0f, 0.0f, -5.0f);

    // Вращаем и рисуем первый домик
    glPushMatrix();
    glEnable(GL_LIGHT1);
    glTranslatef(-1.5f, 0.0f, 0.0f);
    // Устанавливаем параметры света для первого домика
    glLightfv(GL_LIGHT1, GL_POSITION, light_position1);
    float roofColor1[] = { 1.0f, 0.0f, 0.0f }; // Красная крыша
    float wallColor1[] = { 0.0f, 1.0f, 0.0f }; // Зеленые стены
    drawHouse(roofColor1, wallColor1, DIFFUSE);
    glPopMatrix();

    // Вращаем и рисуем третий домик
    glPushMatrix();
    glEnable(GL_LIGHT2);
    glTranslatef(1.5f, 0.0f, 0.0f);
    // Устанавливаем параметры света для третьего домика
    glLightfv(GL_LIGHT2, GL_POSITION, light_position2);
    float roofColor2[] = { 1.0f, 1.0f, 1.0f }; // Белая крыша
    float wallColor2[] = { 0.6f, 0.6f, 0.6f }; // Серые стены
    drawHouse(roofColor2, wallColor2, DIFFUSE);
```

```
    glPopMatrix();

    // Вращаем и рисуем второй домик
    glPushMatrix();
    glEnable(GL_LIGHT3);
    glTranslatef(0.0f, 0.0f, 0.0f);
    // Устанавливаем параметры света для второго домика
    glLightfv(GL_LIGHT3, GL_POSITION, light_position3);
    float roofColor3[] = { 0.0f, 0.0f, 1.0f }; // Синяя крыша
    float wallColor3[] = { 1.0f, 1.0f, 0.0f }; // Желтые стены
    drawHouse(roofColor3, wallColor3, MIRROR);
    glPopMatrix();

    glFlush();
    glutSwapBuffers();
}



void reshape(int w, int h) {
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0, (double)w / (double)h, 1.0, 10.0);
    glMatrixMode(GL_MODELVIEW);
}

void mouse(int button, int state, int x, int y) {
    if (button == GLUT_LEFT_BUTTON) { // Change this line
        if (state == GLUT_DOWN) { // Change this line
            isDragging = true;
            xDragStart = x;
            yDragStart = y;
        }
        else {
            isDragging = false;
        }
    }
}

void motion(int x, int y) {
    if (isDragging) {
        int xdelta = x - xDragStart;
        int ydelta = y - yDragStart;

        light_position1[0] += xdelta * 0.01;
        light_position1[1] -= ydelta * 0.01;

        light_position2[0] += xdelta * 0.01;
        light_position2[1] -= ydelta * 0.01;

        light_position3[0] += xdelta * 0.01;
        light_position3[1] -= ydelta * 0.01;

        xDragStart = x;
        yDragStart = y;

        glutPostRedisplay();
    }
}


int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
```

```
    glutInitWindowSize(800, 600);
    glutCreateWindow("3D Houses");

    init(); // Вызываем функцию инициализации света

    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutMouseFunc(mouse);
    glutMotionFunc(motion);

    glEnable(GL_DEPTH_TEST);

    glutMainLoop();

    return 0;
}
```

# 3. Скриншоты выполнения работы






## 4. Вывод

В данной программе буфер глубины (GL_DEPTH_TEST) используется для правильного определения порядка отображения объектов в трехмерном пространстве, что позволяет избегать неправильного перекрытия объектов. Прозрачность достигается с помощью функции glEnable(GL_BLEND) и glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA), что позволяет настраивать прозрачность для определенных частей объектов, таких как стены и крыши, создавая эффекты зеркального отражения и смешанного отражения. Изучив использование буфера глубины и прозрачности, я освоили методы правильного отображения объектов в трехмерной графике, контроль над прозрачностью элементов сцены, а также настройку параметров освещения и различных типов отражения.